

TECHNICAL DOCUMENTATION

Overlayd-AI

Technical Architecture & Implementation Guide

Offline AI-Powered Android Device Control

System Architecture Deep Dive



Table of Contents

1. The Core Problem: Android Sandbox Architecture
2. Breaking the Sandbox: Shizuku & Rish
3. The Brain: llama.cpp Inference Engine
4. The Logic Glue: Node.js & Telegram Bridge
5. Vision Integration: OpenClaw & Multimodal AI
6. System Architecture Overview
7. Executive Summary

1 The Core Problem: Android Sandbox Architecture

Understanding the Constraint

Android's security model places every application inside an isolated "Sandbox" environment. This is a fundamental security feature designed to prevent malicious applications from accessing system resources or interfering with other apps.

Termux, while powerful as a Linux environment emulator, operates within these same constraints. Any process running inside Termux—including AI models—is fundamentally trapped within this sandbox boundary. This creates a critical limitation:

- **Screen Access:** AI cannot read or interact with the device's display
- **App Control:** Cannot launch, close, or manipulate other applications
- **System Settings:** Cannot toggle WiFi, Bluetooth, or modify system preferences
- **Hardware Control:** Limited access to sensors, cameras, and other hardware components

To enable true AI-driven device automation, we must architect a solution that "breaks out" of this sandbox while maintaining system stability and security.

2 Breaking the Sandbox: Shizuku & Rish

2.1 The Privilege Escalation Mechanism

This is where the core innovation of Overlayd-AI resides. We leverage Android's built-in developer infrastructure to achieve system-level access without requiring root privileges.



Shizuku

PRIVILEGE BROKER

An application that utilizes Android's "Wireless Debugging" feature to acquire ADB (Android Debug Bridge) permissions without root access.



Rish

COMMAND WRAPPER

A special shell command exported from Shizuku into Termux that executes commands with elevated system privileges.

2.2 The "God Mode" Execution Model

When we export the necessary files from Shizuku into the Termux environment, we gain access to the `rish` command. Any command wrapped within `rish` executes with system-level privileges:

```
# Example: Force home screen navigation
rish -c "input keyevent 3"

# Example: Launch YouTube application
rish -c "monkey -p com.google.android.youtube 1"

# Example: Toggle WiFi state
rish -c "svc wifi disable"
```

The Bridge is Established

This creates our critical bridge out of Termux. The AI, running within the sandbox, can now execute commands that directly manipulate the Android system through the `rish` tunnel.

3 The Brain: llama.cpp Inference Engine

3.1 Performance-First Architecture Decision

Traditional AI implementations on mobile devices rely on Python-based frameworks or cloud API calls. Both approaches present significant limitations:

Approach	Limitation	Our Solution
Python Frameworks	Interpreted execution, high memory overhead	Native C/C++ compilation
Cloud APIs	Requires internet, latency issues, privacy concerns	Local inference server
Generic Binaries	Suboptimal performance on target architecture	Architecture-specific compilation

3.2 llama.cpp: Pure C/C++ Implementation

Our solution automatically downloads and compiles **llama.cpp**, a high-performance inference engine written in pure C/C++. The compilation process uses `cmake` to target the device's specific processor architecture, achieving mathematically optimal execution speed.

OpenAI-Compatible Local Server

We configure llama.cpp to run as a persistent server on `localhost:8080`, exposing an OpenAI-compatible API endpoint. This allows any component in our stack to communicate with the AI using standard HTTP requests.

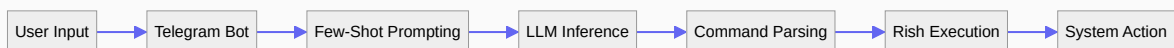
```
# Server startup configuration
./server -m model.gguf \
  --host 127.0.0.1 \
  --port 8080 \
  -c 4096 \
  --timeout 300
```

4 The Logic Glue: Node.js & Telegram Bridge

4.1 The Communication Pipeline

To map user chat messages to Shizuku commands, we implemented a lightweight JavaScript bridge using Node.js (`telegram_bot.js`). This component serves as the orchestration layer between natural language input and system-level execution.

Command Processing Flow



4.2 Few-Shot Prompting Strategy

The bridge uses carefully engineered few-shot prompting to guide the local AI in generating structured command responses:

1

User Input Capture

Telegram bot receives natural language command (e.g., "Open YouTube")

2 Contextual Prompt Construction

Node.js constructs a prompt with examples mapping intents to ADB commands

3 LLM Inference

Local AI processes the prompt and returns structured response

4 Command Extraction

Response parsed for `CMD:` prefix, raw command extracted

5 Privileged Execution

Command wrapped in `rish` tunnel and executed with system privileges

4.3 Example Execution Trace

```
# User sends: "Open YouTube"

# AI Response Format:
CMD: monkey -p com.google.android.youtube 1

# Node.js Processing:
const command = aiResponse.replace("CMD:", "").trim();
const rishCommand = `rish -c "${command}"`;
exec(rishCommand);

# Result: YouTube launches on device
```

Achievement: Fully Offline AI Control

The AI successfully opened YouTube using only local resources—no internet connection required. The entire inference pipeline runs natively on the device.

5 Vision Integration: OpenClaw & Multimodal AI

5.1 Visual Understanding Capability

Qwen is a multimodal vision model, meaning it can process and understand visual information in addition to text. This unlocks a critical capability: the AI can "see" the screen and make decisions based on visual context.

5.2 OpenClaw Integration

We integrate **OpenClaw**, a third-party automation framework that provides:

- **Rapid Screenshot Capture:** High-frequency screen capture for real-time visual analysis
- **UI Layout Analysis:** Extraction of interactive elements and their coordinates
- **Element Detection:** Identification of buttons, text fields, and other UI components

5.3 The OpenClaw-Local Bridge

The Challenge

OpenClaw is designed to connect to ChatGPT over the internet. This violates our offline-first requirement.

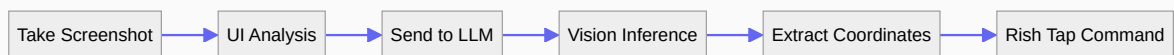
Our solution: a custom executable called `openclaw-local` that intercepts OpenClaw's environment paths and redirects all API calls to our local server:


```
# Intercepted endpoint configuration
BASE_URL = "http://127.0.0.1:8080/v1"
API_KEY = "sk-local-offline" # Dummy key for compatibility

# All OpenAI-compatible endpoints redirected:
# - /v1/chat/completions
# - /v1/models
# - /v1/embeddings
```

5.4 Vision-to-Action Pipeline

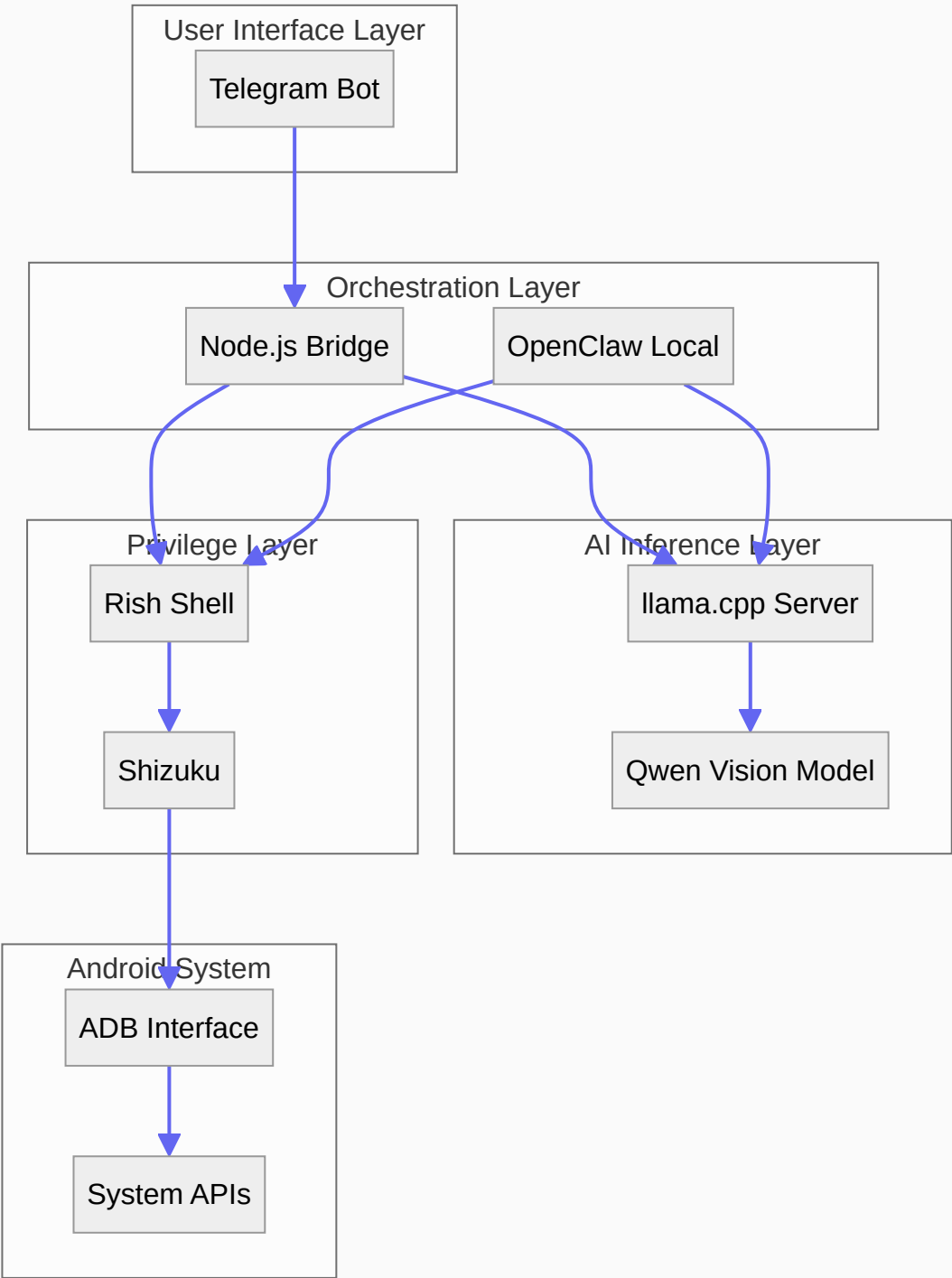
Visual Automation Flow



The result is a closed-loop system where OpenClaw captures the screen, funnels the visual data entirely offline into the Llama engine, determines the location of target UI elements, and uses `rish` to tap them natively.

6 System Architecture Overview

Complete System Architecture



Layer	Component	Function
-------	-----------	----------

User Interface	Telegram Bot	Natural language input/output interface
Orchestration	Node.js Bridge	Prompt engineering, command parsing, execution flow
Orchestration	OpenClaw Local	Visual capture, UI analysis, coordinate extraction
AI Inference	llama.cpp + Qwen	Local LLM inference, multimodal understanding
Privilege	Shizuku + Rish	ADB privilege acquisition, command execution
System	Android APIs	Hardware control, app management, settings

7 Executive Summary

THE COMPLETE PICTURE

"We are deploying a lightweight C++ AI inference server directly on the mobile device, communicating with it through a Telegram bot interface running in the background. By granting this system Shizuku (Developer) privileges, the AI can break out of the command terminal sandbox and natively control phone settings, applications, and hardware—all while completely disconnected from the internet."

Key Technical Achievements

01 100% Offline Operation PRIVACY FIRST

No data leaves the device. All inference happens locally using the mobile processor.

02 Native Performance C++ OPTIMIZED

Architecture-specific compilation ensures maximum inference speed on mobile hardware.

03 No Root Required DEVELOPER ACCESS

Leverages official Android debugging features—no device modification needed.

04 Vision Capable MULTIMODAL AI

Can see, understand, and interact with on-screen elements through visual analysis.

Technical Confidence for Your Video

This architecture represents a sophisticated integration of multiple technologies: privilege escalation through official Android APIs, native C++ performance optimization, modern JavaScript orchestration, and multimodal AI inference. The result is a system that appears to perform "magic" while being built on solid, explainable engineering principles.